

```
'use strict';
```

```
const { findProject } = require('./guild-project-data.js');
const { logEconomyEvent, EVENT_TYPES } = require('../lib/economy-audit.js');
const { logWorldEvent } = require('../lib/world-feed.js');
const { notifyGuildMembers } = require('../lib/notifications.js');
```

```
/**
```

```
 * GUILD PROJECTS — движок. Тот же принцип, что и весь остальной проект:
 * изменения текущего игрока (списание из его личного вклада учитывается
 * через store) — обычная работа через store, изменения общего прогресса
 * проекта — атомарная операция в guild-store-upstash.js (contributeToProjectAtom
 * чтобы двое участников, вносящих ресурсы одновременно, не могли
 * "перевыполнить" требование дважды или потерять чей-то вклад.
 */
```

```
class GuildProjectError extends Error {
  constructor(code) { super(code); this.name = 'GuildProjectError'; this.code = c
}
```

```
/** Начинает проект — можно вести НЕСКОЛЬКО проектов параллельно (в
 * отличие от гильд-апгрейдов, где путь линейный), поэтому просто
 * создаёт запись, без проверки "уже что-то идёт". Только офицер/лидер —
 * та же роль, что и для гильд-апгрейда. */
```

```
async function startProject(deps, player, projectId) {
  const { store } = deps;
  if (!player.guildId) throw new GuildProjectError('NOT_IN_GUILD');
  const project = findProject(projectId);
  if (!project) throw new GuildProjectError('PROJECT_NOT_FOUND');

  const role = await store.getGuildMemberRole(player.guildId, player.id);
  if (role !== 'leader' && role !== 'officer') throw new GuildProjectError('NOT_O

  const existing = await store.getGuildProject(player.guildId, projectId);
  if (existing && existing.status === 'active') throw new GuildProjectError('ALRE

  await store.startGuildProject(player.guildId, projectId, project.requirements);
  return { project };
}
```

```
/** Вносит вклад РЕСУРСОМ из личного инвентаря игрока в проект гильдии —
 * снятие с игрока обычная мутация (это его стак), запись в прогресс
 * проекта атомарная. contribution points начисляются тем же вызовом,
 * для личного рейтинга (не для награды — см. заметку в data-файле). */
```

```

async function contributeResource(deps, player, projectId, resource, tier, qty) {
  const { store } = deps;
  if (!player.guildId) throw new GuildProjectError('NOT_IN_GUILD');
  const project = findProject(projectId);
  if (!project) throw new GuildProjectError('PROJECT_NOT_FOUND');

  const stack = (player.inventory || []).find((i) => i.resource === resource && i
  if (!stack || stack.qty < qty) throw new GuildProjectError('INSUFFICIENT_RESOUR

  const need = project.requirements.resources.find((r) => r.resource === resource
  if (!need) throw new GuildProjectError('RESOURCE_NOT_NEEDED');

  stack.qty -= qty;
  player.inventory = player.inventory.filter((i) => i.qty > 0);

  const result = await store.contributeToProjectAtomic(player.guildId, projectId,
  logEconomyEvent(deps, { type: EVENT_TYPES.GUILD_DONATE, playerId: player.id, re
  return { player, progress: result };
}

/** Вносит вклад КРЕДИТАМИ. */
async function contributeCredits(deps, player, projectId, amount) {
  const { store } = deps;
  if (!player.guildId) throw new GuildProjectError('NOT_IN_GUILD');
  if (amount <= 0 || (player.credits || 0) < amount) throw new GuildProjectError(
  const project = findProject(projectId);
  if (!project) throw new GuildProjectError('PROJECT_NOT_FOUND');

  player.credits -= amount;
  const result = await store.contributeToProjectAtomic(player.guildId, projectId,
  logEconomyEvent(deps, { type: EVENT_TYPES.GUILD_DONATE, playerId: player.id, cr
  return { player, progress: result };
}

/** Проверяет, выполнены ли ВСЕ требования проекта – если да, завершает
 * его атомарно (переводит status в 'completed', эффект начинает
 * действовать сразу для всех участников гильдии через
 * getActiveGuildProjectEffects). Может вызвать любой участник – сама
 * проверка идемпотентна (повторный вызов после завершения просто вернёт
 * ALREADY_COMPLETED, не ломает состояние). */
async function tryCompleteProject(deps, player, projectId) {
  const { store } = deps;
  const project = findProject(projectId);
  if (!project) throw new GuildProjectError('PROJECT_NOT_FOUND');
  const progress = await store.getGuildProject(player.guildId, projectId);
  if (!progress) throw new GuildProjectError('PROJECT_NOT_FOUND');
  if (progress.status === 'completed') return { alreadyCompleted: true, project }

```

```

const creditsOk = progress.creditsContributed >= project.requirements.credits;
const resourcesOk = project.requirements.resources.every((need) => {
  const have = progress.resourcesContributed.find((r) => r.resource === need.re
  return have && have.qty >= need.qty;
});
if (!creditsOk || !resourcesOk) return { alreadyCompleted: false, project, comp

const completed = await store.completeGuildProjectAtomic(player.guildId, projec
if (!completed.success) return { alreadyCompleted: false, project, complete: fa

const guild = await store.getGuild(player.guildId).catch(() => null);
if (guild) {
  logWorldEvent(deps, { type: 'guild_project_completed', text: `Гильдия «${guil
  const memberIds = await store.getGuildMemberIds(player.guildId).catch(() => [
  notifyGuildMembers(deps, memberIds, `🚧 Проект «${project.name}» завершён! ${
}

  return { alreadyCompleted: false, project, complete: true };
}

/** Суммарные бонусы от ВСЕХ завершённых проектов гильдии — читать здесь
 * же, где раньше читали activeGuildBonuses(guild-levels.js), это
 * ОТДЕЛЬНЫЙ, дополнительный источник бонусов (проекты и уровни не
 * конкурируют, складываются). */
async function getActiveGuildProjectEffects(deps, guildId) {
  const { store } = deps;
  if (!guildId) return { repairDiscountPct: 0, rareDiscoveryBonusPct: 0, ambushRi
  const completedIds = await store.getCompletedGuildProjectIds(guildId);
  const merged = { repairDiscountPct: 0, rareDiscoveryBonusPct: 0, ambushRiskRedu
  for (const id of completedIds) {
    const project = findProject(id);
    if (!project) continue;
    if (project.effect.type === 'repair_discount') merged.repairDiscountPct += pr
    if (project.effect.type === 'rare_discovery') merged.rareDiscoveryBonusPct +=
    if (project.effect.type === 'ambush_risk_reduction') merged.ambushRiskReducti
  }
  return merged;
}

module.exports = {
  GuildProjectError, startProject, contributeResource, contributeCredits,
  tryCompleteProject, getActiveGuildProjectEffects,
};

```